

(Somaiya Vidyavihar University)

Department of Computer Engineering

Batch: A1

Roll No.: 16010123012

Experiment / Assignment / Tutorial No.: 02

Title: Implementation of linear Regression.

Aim: To understand & implement linear regression using python.

Expected Outcome of Experiment:

CO2: Describe and apply supervised learning methods.

Books/ Journals/ Websites referred:

<https://www.ibm.com/topics/linearregression#:~:text=Resources,What%20is%20linear%20regression%3F,is%20called%20the%20independent%20variable.>

<https://www.geeksforgeeks.org/ml-linear-regression/>

<https://towardsdatascience.com/understand-the-data-with-univariate-and-multivariate-charts-and-plots-in-python-3b9fcd68cd8>

Theory of Linear Regression:

Linear regression analysis is used to predict the value of a variable based on the value of another variable. The variable you want to predict is called the dependent variable. The variable you are using to predict the other variable's value is called the independent variable. Linear Regression is a machine learning algorithm based on supervised learning. It performs a regression task. Regression models a target prediction value based on independent variables. It is mostly used for finding out the relationship between variables and forecasting. Different regression models differ based on – the kind of relationship between dependent and independent variables they are considering, and the number of independent variables getting used.

Problem Definition:

Apply linear regression on diabetes dataset or any other dataset and fit the regression model on the data using gradient descent algorithm. Also compute error values if any.

(Somaiya Vidyavihar University)

Department of Computer Engineering

```
import pandas as pd
df=pd.read_csv("/content/Training.csv")
display(df)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1
...	...	...	...	...	...	...	...	...	...
2455	3	126	88	41	235	39.3	0.704	27	0
2456	4	123	62	0	0	32.0	0.226	35	1
2457	1	80	74	11	60	30.0	0.527	22	0
2458	1	96	64	27	87	33.2	0.289	21	0
2459	6	105	70	32	68	30.8	0.122	37	0

2460 rows × 9 columns

```
import numpy as np
skin_thickness_mean = df[df['SkinThickness'] != 0]['SkinThickness'].mean()
df['SkinThickness'] = df['SkinThickness'].replace(0, skin_thickness_mean)
display(df)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35.000000	0	33.6	0.627	50	1
1	1	85	66	29.000000	0	26.6	0.351	31	0
2	8	183	64	28.977051	0	23.3	0.672	32	1
3	1	89	66	23.000000	94	28.1	0.167	21	0
4	0	137	40	35.000000	168	43.1	2.288	33	1
...	...	...	...	...	...	...	...	...	...
2455	3	126	88	41.000000	235	39.3	0.704	27	0
2456	4	123	62	28.977051	0	32.0	0.226	35	1
2457	1	80	74	11.000000	60	30.0	0.527	22	0
2458	1	96	64	27.000000	87	33.2	0.289	21	0
2459	6	105	70	32.000000	68	30.8	0.122	37	0

2460 rows × 9 columns

```
import statsmodels.api as sm

X = df['Glucose'].values.reshape(-1, 1)
y = df['BMI'].values

X = sm.add_constant(X)

print("Shape of X after adding constant:", X.shape)
print("Shape of y:", y.shape)
print("First 5 rows of X:\n", X[:5])
print("First 5 rows of y:\n", y[:5])

Shape of X after adding constant: (2460, 2)
Shape of y: (2460,)
First 5 rows of X:
[[ 1. 148.]
 [ 1.  85.]
 [ 1. 183.]
 [ 1.  89.]
 [ 1. 137.]]
First 5 rows of y:
[33.6 26.6 23.3 28.1 43.1]

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Shape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of y_test:", y_test.shape)

Shape of X_train: (1968, 2)
Shape of X_test: (492, 2)
Shape of y_train: (1968,)
Shape of y_test: (492,)

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

model = LinearRegression()

model.fit(X_train, y_train)

predictions_test_sklearn = model.predict(X_test)

print(f"Intercept (Bias): {model.intercept_:.4f}")
print(f"Coefficient (Weight for Glucose): {model.coef_[0]:.4f}")

*** Intercept (Bias): 24.1781
Coefficient (Weight for Glucose): 0.0000
```

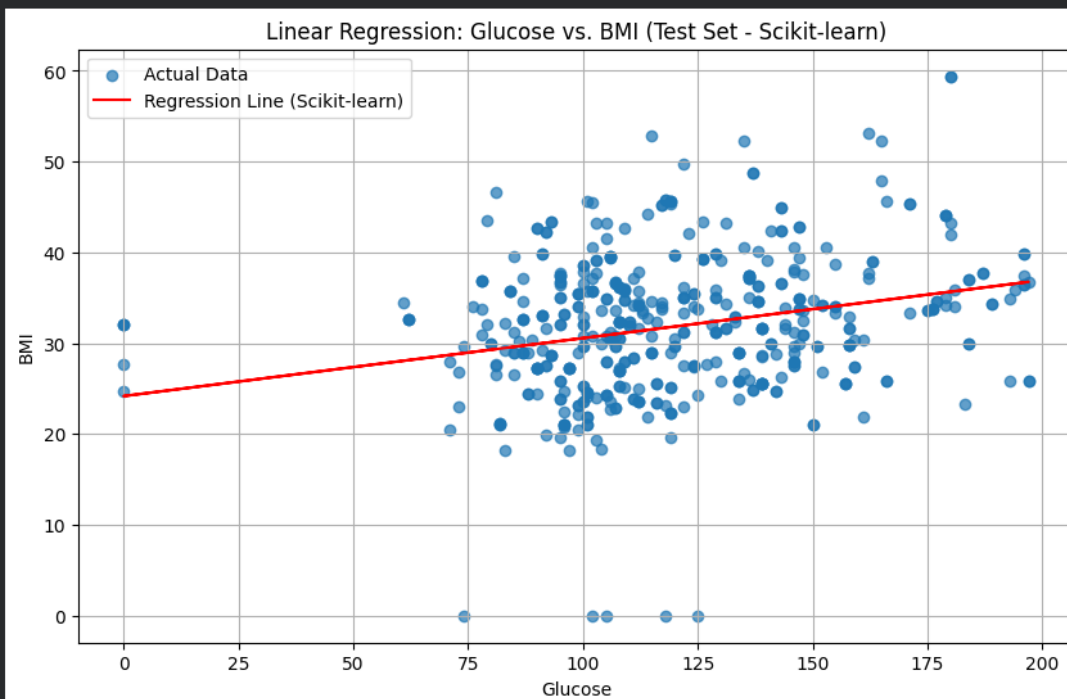
(Somaiya Vidyavihar University)

Department of Computer Engineering

```
mse_test_sklearn = mean_squared_error(y_test, predictions_test_sklearn)  
  
print(f"Mean Squared Error on Test Set (Scikit-learn): {mse_test_sklearn:.4f}")
```

Mean Squared Error on Test Set (Scikit-learn): 54.6077

```
import matplotlib.pyplot as plt  
  
plt.figure(figsize=(10, 6))  
plt.scatter(X_test[:, 1], y_test, label='Actual Data', alpha=0.7)  
plt.plot(X_test[:, 1], predictions_test_sklearn, color='red', label='Regression Line (Scikit-learn)')  
  
plt.xlabel('Glucose')  
plt.ylabel('BMI')  
plt.title('Linear Regression: Glucose vs. BMI (Test Set - Scikit-learn)')  
plt.legend()  
plt.grid(True)  
plt.show()
```



(Somaiya Vidyavihar University)

Department of Computer Engineering

```
df_numeric = df.select_dtypes(include=['number'])
correlation_matrix = df_numeric.corr()
display(correlation_matrix)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
Pregnancies	1.000000	0.185535	0.110625	-0.029415	-0.030863	0.094674	-0.053568	0.541545	0.218263
Glucose	0.185535	1.000000	0.116223	0.058602	0.351118	0.252411	0.167317	0.270796	0.440045
BloodPressure	0.110625	0.116223	1.000000	0.205257	0.102433	0.245112	0.007001	0.202828	0.069604
SkinThickness	-0.029415	0.058602	0.205257	1.000000	0.478468	0.376467	0.173692	-0.130043	0.071114
Insulin	-0.030863	0.351118	0.102433	0.478468	1.000000	0.204730	0.228555	-0.017574	0.109022
BMI	0.094674	0.252411	0.245112	0.376467	0.204730	1.000000	0.123726	0.067208	0.308345
DiabetesPedigreeFunction	-0.053568	0.167317	0.007001	0.173692	0.228555	0.123726	1.000000	0.041072	0.160543
Age	0.541545	0.270796	0.202828	-0.130043	-0.017574	0.067208	0.041072	1.000000	0.217349
Outcome	0.218263	0.440045	0.069604	0.071114	0.109022	0.308345	0.160543	0.217349	1.000000

```
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

X_multi = df[['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin']].values
y_multi = df['BMI'].values

X_multi = sm.add_constant(X_multi)

print("Shape of X_multi after adding constant:", X_multi.shape)
print("Shape of y_multi:", y_multi.shape)
print("First 5 rows of X_multi:\n", X_multi[:5])
print("First 5 rows of y_multi:\n", y_multi[:5])

Shape of X_multi after adding constant: (2460, 5)
Shape of y_multi: (2460,)
First 5 rows of X_multi:
[[ 1. 148. 72. 35.  0.]
 [ 1.  85. 66. 29.  0.]
 [ 1. 183. 64.  0.  0.]
 [ 1.  89. 66. 23. 94.]
 [ 1. 137. 40. 35. 168.]]
First 5 rows of y_multi:
[33.6 26.6 23.3 28.1 43.1]

X_multi_train, X_multi_test, y_multi_train, y_multi_test = train_test_split(X_multi, y_multi, test_size=0.2, random_state=42)

model_multi = LinearRegression()
model_multi.fit(X_multi_train, y_multi_train)

predictions_multi = model_multi.predict(X_multi_test)

mse_multi = mean_squared_error(y_multi_test, predictions_multi)

print(f"Mean Squared Error on Test Set (Multiple Linear Regression): {mse_multi:.4f}")
print(f"Intercept (Bias): {model_multi.intercept_:.4f}")

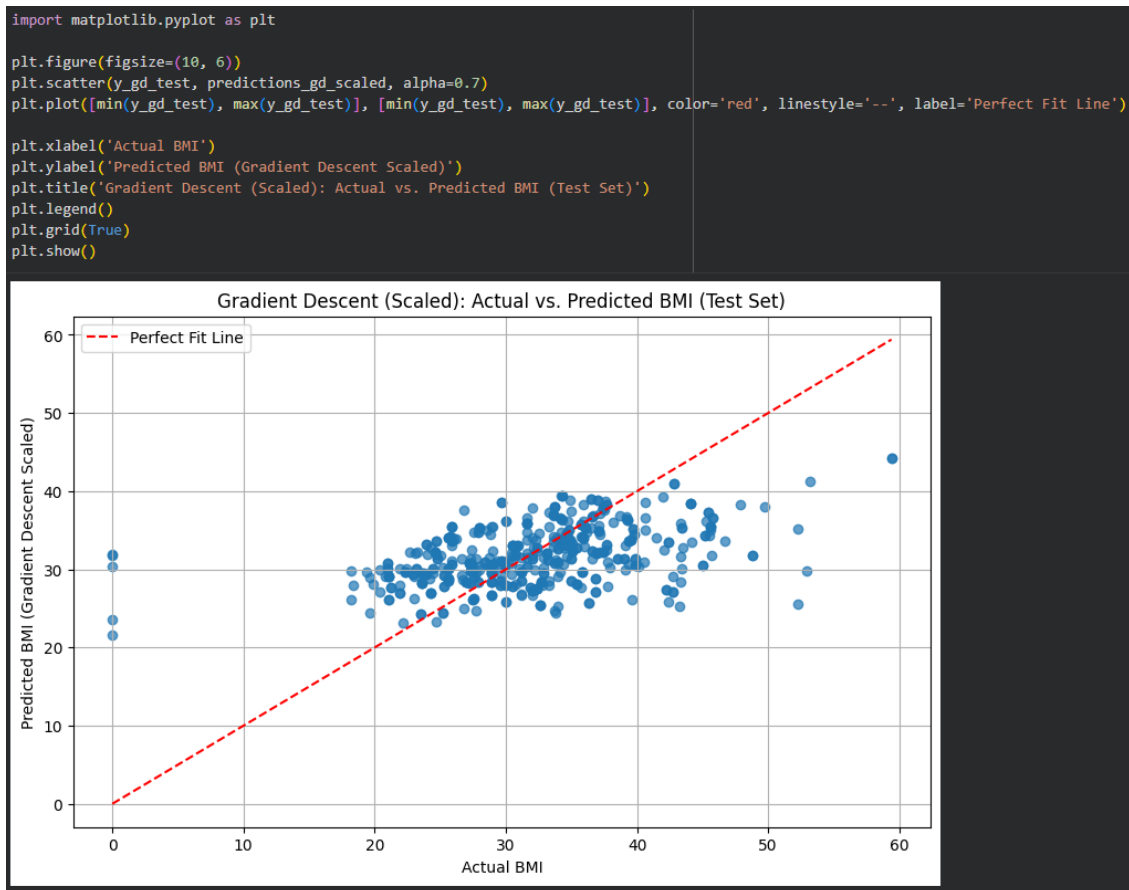
feature_names = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin']
for i, coef in enumerate(model_multi.coef_[1:]):
    print(f"Coefficient for {feature_names[i]}: {coef:.4f}")

Mean Squared Error on Test Set (Multiple Linear Regression): 45.6466
Intercept (Bias): 16.9505
Coefficient for Glucose: 0.0612
Coefficient for BloodPressure: 0.0603
Coefficient for SkinThickness: 0.1845
Coefficient for Insulin: -0.0052
```



(Somaiya Vidyavihar University)

Department of Computer Engineering



API/Function used:

- `numpy.linspace()` – to generate evenly spaced synthetic input values
- `numpy.random.normal()` – to add Gaussian noise
- `sklearn.linear_model.LinearRegression` – to fit regression model using normal equation
- `sklearn.metrics.r2_score()` – to evaluate goodness of fit
- `sklearn.metrics.mean_squared_error()` – to compute RMSE
- `matplotlib.pyplot.scatter()` and `plot()` – for data visualization
- Custom Gradient Descent implementation using NumPy matrix operations
- `reshape()` – to convert 1D data into 2D feature matrix

Conclusion:

In this experiment, Linear Regression was implemented using both the sklearn library and Gradient Descent techniques. Synthetic linear and nonlinear datasets were generated to analyze model behavior. Batch, Stochastic, and Mini-Batch Gradient Descent were

(Somaiya Vidyavihar University)

Department of Computer Engineering

applied, and all methods converged to similar parameter values with comparable error metrics. Linear regression performed accurately on linear data but showed limitations on nonlinear patterns. This experiment helped understand regression modeling, optimization methods, and the impact of data structure on model performance.

Post Lab:

1. Generate synthetic dataset (linear and nonlinear both) for e.g. $y = (x/2) + \sin(x) +$ Gaussian noise or take any function of your choice.

```
import numpy as np
import pandas as pd

x_linear = np.linspace(0, 10, 100)

y_linear = 2 * x_linear + 5

y_linear_noisy = y_linear + np.random.normal(0, 2, 100)

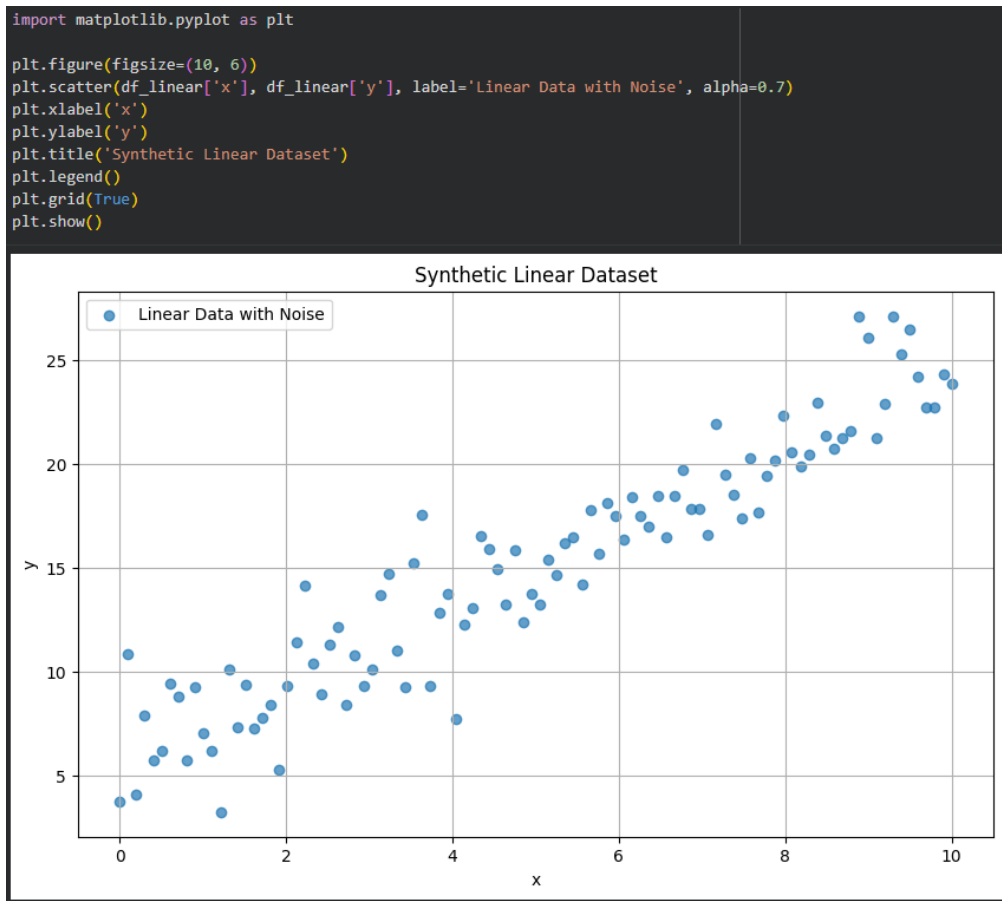
df_linear = pd.DataFrame({'x': x_linear, 'y': y_linear_noisy})

display(df_linear.head())
```

	x	y
0	0.00000	3.714047
1	0.10101	10.846415
2	0.20202	4.055685
3	0.30303	7.882882
4	0.40404	5.705729

(Somaiya Vidyavihar University)

Department of Computer Engineering



```
import numpy as np
import pandas as pd

x_nonlinear = np.linspace(0, 10, 100)

y_nonlinear = x_nonlinear**2 + 5

y_nonlinear_noisy = y_nonlinear + np.random.normal(0, 5, 100)

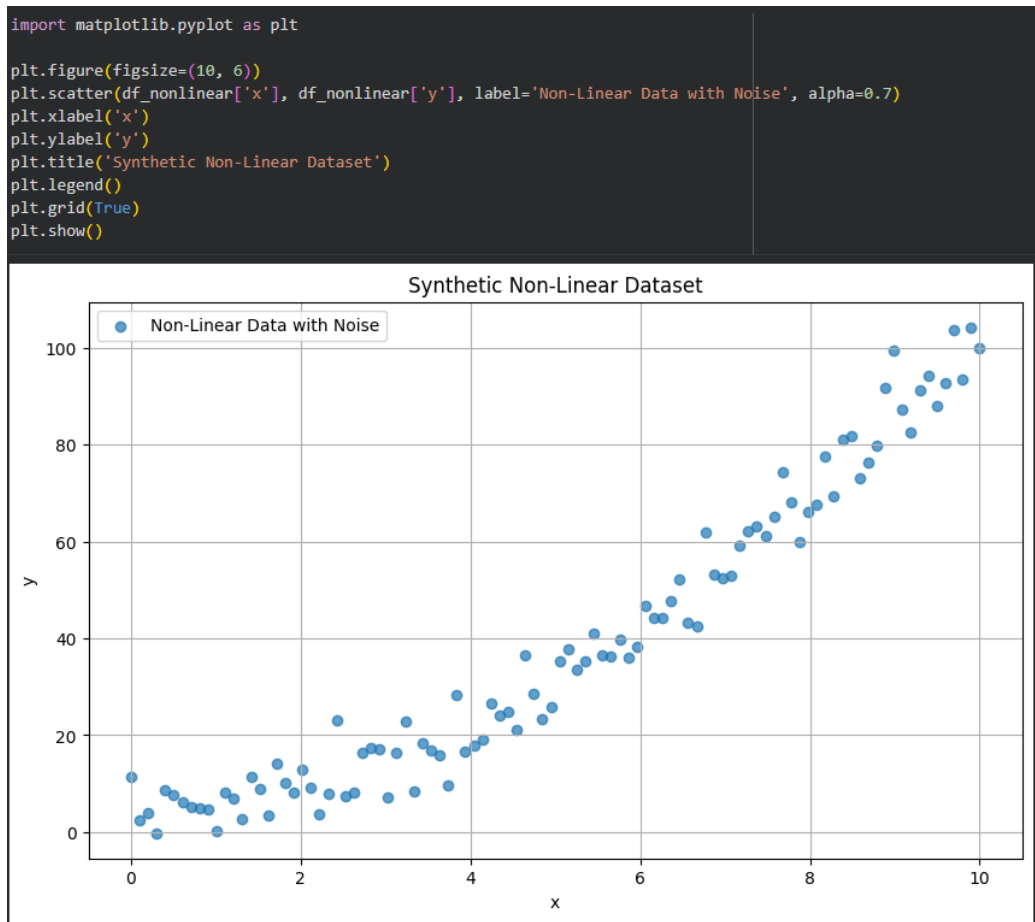
df_nonlinear = pd.DataFrame({'x': x_nonlinear, 'y': y_nonlinear_noisy})

display(df_nonlinear.head())
```

	x	y
0	0.00000	11.330688
1	0.10101	2.361421
2	0.20202	4.068176
3	0.30303	-0.205931
4	0.40404	8.568887

(Somaiya Vidyavihar University)

Department of Computer Engineering



2. Fit the regression model using sklearn library.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X_linear_skl = df_linear['x'].values.reshape(-1, 1)
y_linear_skl = df_linear['y'].values

X_linear_skl_train, X_linear_skl_test, y_linear_skl_train, y_linear_skl_test = train_test_split(X_linear_skl, y_linear_skl, test_size=0.2, random_state=42)

model_linear_skl = LinearRegression()

model_linear_skl.fit(X_linear_skl_train, y_linear_skl_train)

predictions_linear_skl = model_linear_skl.predict(X_linear_skl_test)

mse_linear_skl = mean_squared_error(y_linear_skl_test, predictions_linear_skl)

print(f"Intercept: {model_linear_skl.intercept_:.4f}")
print(f"Coefficient: {model_linear_skl.coef_[0]:.4f}")
print(f"Mean Squared Error (Linear Regression on Synthetic Linear Data): {mse_linear_skl:.4f}")

Intercept: 5.5648
Coefficient: 1.9186
Mean Squared Error (Linear Regression on Synthetic Linear Data): 3.8408
```

(Somaiya Vidyavihar University)

Department of Computer Engineering

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X_nonlinear_poly = df_nonlinear['x'].values.reshape(-1, 1)
y_nonlinear_poly = df_nonlinear['y'].values

poly = PolynomialFeatures(degree=2)
X_nonlinear_poly_transformed = poly.fit_transform(X_nonlinear_poly)

X_nonlinear_poly_train, X_nonlinear_poly_test, y_nonlinear_poly_train, y_nonlinear_poly_test = train_test_split(X_nonlinear_poly_transformed, y_nonlinear_poly, test_size=0.2, random_state=42)

model_poly_skl = LinearRegression()

model_poly_skl.fit(X_nonlinear_poly_train, y_nonlinear_poly_train)

predictions_poly_skl = model_poly_skl.predict(X_nonlinear_poly_test)

mse_poly_skl = mean_squared_error(y_nonlinear_poly_test, predictions_poly_skl)

print(f"Intercept (Polynomial Regression): {model_poly_skl.intercept_:.4f}")
print(f"Coefficients (Polynomial Regression): {model_poly_skl.coef_}")
print(f"Mean Squared Error (Polynomial Regression on Synthetic Non-Linear Data): {mse_poly_skl:.4f}")

Intercept (Polynomial Regression): 3.9445
Coefficients (Polynomial Regression): [0.          1.11621368  0.8794806 ]
Mean Squared Error (Polynomial Regression on Synthetic Non-Linear Data): 26.9974
```

3. Fit the regression model on the data using gradient descent algorithm and its different variants.

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error
import numpy as np
from sklearn.model_selection import train_test_split

X_linear_gd = df_linear['x'].values.reshape(-1, 1)
y_linear_gd = df_linear['y'].values

X_linear_gd_train, X_linear_gd_test, y_linear_gd_train, y_linear_gd_test = train_test_split(X_linear_gd, y_linear_gd, test_size=0.2, random_state=42)

scaler_linear_gd = StandardScaler()

X_linear_gd_train_scaled = scaler_linear_gd.fit_transform(X_linear_gd_train)
X_linear_gd_test_scaled = scaler_linear_gd.transform(X_linear_gd_test)

sgd_linear_model_scaled = SGDRegressor(loss='squared_error', max_iter=1000, tol=1e-3, random_state=42)

sgd_linear_model_scaled.fit(X_linear_gd_train_scaled, y_linear_gd_train)

predictions_linear_gd_scaled = sgd_linear_model_scaled.predict(X_linear_gd_test_scaled)

mse_linear_gd_scaled = mean_squared_error(y_linear_gd_test, predictions_linear_gd_scaled)
print(f"Mean Squared Error (SGDRegressor on Synthetic Linear Data): {mse_linear_gd_scaled:.4f}")

rmse_linear_gd_scaled = np.sqrt(mse_linear_gd_scaled)
print(f"Root Mean Squared Error (SGDRegressor on Synthetic Linear Data): {rmse_linear_gd_scaled:.4f}")

print(f"Intercept (SGDRegressor): {sgd_linear_model_scaled.intercept_[0]:.4f}")
print(f"Coefficient (SGDRegressor): {sgd_linear_model_scaled.coef_[0]:.4f}")

Mean Squared Error (SGDRegressor on Synthetic Linear Data): 3.8046
Root Mean Squared Error (SGDRegressor on Synthetic Linear Data): 1.9505
Intercept (SGDRegressor): 15.3578
Coefficient (SGDRegressor): 5.5775
```

(Somaiya Vidyavihar University)

Department of Computer Engineering

```
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import SGDRegressor
from sklearn.metrics import mean_squared_error
import numpy as np
from sklearn.model_selection import train_test_split

X_nonlinear_gd = df_nonlinear['x'].values.reshape(-1, 1)
y_nonlinear_gd = df_nonlinear['y'].values

poly_gd = PolynomialFeatures(degree=2)
X_nonlinear_gd_poly = poly_gd.fit_transform(X_nonlinear_gd)

X_nonlinear_gd_train, X_nonlinear_gd_test, y_nonlinear_gd_train, y_nonlinear_gd_test = train_test_split(X_nonlinear_gd_poly, y_nonlinear_gd, test_size=0.2, random_state=42)

scaler_nonlinear_gd = StandardScaler()
X_nonlinear_gd_train_scaled = scaler_nonlinear_gd.fit_transform(X_nonlinear_gd_train)
X_nonlinear_gd_test_scaled = scaler_nonlinear_gd.transform(X_nonlinear_gd_test)

sgd_nonlinear_model_scaled = SGDRegressor(loss='squared_error', max_iter=1000, tol=1e-3, random_state=42)
sgd_nonlinear_model_scaled.fit(X_nonlinear_gd_train_scaled, y_nonlinear_gd_train)

predictions_nonlinear_gd_scaled = sgd_nonlinear_model_scaled.predict(X_nonlinear_gd_test_scaled)

mse_nonlinear_gd_scaled = mean_squared_error(y_nonlinear_gd_test, predictions_nonlinear_gd_scaled)
rmse_nonlinear_gd_scaled = np.sqrt(mse_nonlinear_gd_scaled)

print(f"Mean Squared Error (SGDRegressor on Synthetic Non-Linear Data): {mse_nonlinear_gd_scaled:.4f}")
print(f"Root Mean Squared Error (SGDRegressor on Synthetic Non-Linear Data): {rmse_nonlinear_gd_scaled:.4f}")

print(f"Intercept (SGDRegressor): {sgd_nonlinear_model_scaled.intercept_[0]:.4f}")
print(f"Coefficients (SGDRegressor for [1, x, x^2]): {sgd_nonlinear_model_scaled.coef_}")

Mean Squared Error (SGDRegressor on Synthetic Non-Linear Data): 30.0924
Root Mean Squared Error (SGDRegressor on Synthetic Non-Linear Data): 5.4857
Intercept (SGDRegressor): 40.2568
Coefficients (SGDRegressor for [1, x, x^2]): [ 0.          5.97784841 24.2683679 ]
```

4. Calculate parameters and compare error values.

```
import pandas as pd
import numpy as np

rmse_linear_skl = np.sqrt(mse_linear_skl)
rmse_poly_skl = np.sqrt(mse_poly_skl)

results = {
    'Model': [
        'Scikit-learn Linear Regression (Linear Data)',
        'Scikit-learn Polynomial Regression (Non-Linear Data)',
        'SGDRegressor (Linear Data, Scaled)',
        'SGDRegressor (Non-Linear Data, Poly + Scaled)'
    ],
    'MSE': [
        mse_linear_skl,
        mse_poly_skl,
        mse_linear_gd_scaled,
        mse_nonlinear_gd_scaled
    ],
    'RMSE': [
        rmse_linear_skl,
        rmse_poly_skl,
        rmse_linear_gd_scaled,
        rmse_nonlinear_gd_scaled
    ],
    'Intercept': [
        model_linear_skl.intercept_,
        model_poly_skl.intercept_,
        sgd_linear_model_scaled.intercept_[0],
        sgd_nonlinear_model_scaled.intercept_[0]
    ],
    'Coefficients': [
        model_linear_skl.coef_[0],
        model_poly_skl.coef_,
        sgd_linear_model_scaled.coef_[0],
        sgd_nonlinear_model_scaled.coef_
    ]
}

comparison_df = pd.DataFrame(results)
display(comparison_df)
```

	Model	MSE	RMSE	Intercept	Coefficients
0	Scikit-learn Linear Regression (Linear Data)	3.840766	1.959787	5.564805	1.918552
1	Scikit-learn Polynomial Regression (Non-Linear...)	26.997428	5.195905	3.944456	[0.0, 1.1162136801491367, 0.8794005969221794]
2	SGDRegressor (Linear Data, Scaled)	3.804586	1.950535	15.357807	5.577515
3	SGDRegressor (Non-Linear Data, Poly + Scaled)	30.092420	5.485656	40.256841	[0.0, 5.977848409215439, 24.268367895928982]